



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ  
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

---

Ανάλυση Δεδομένων Εγκληματικότητας  
στο Los Angeles με Apache Spark & Hadoop

---

Διαχείριση Δεδομένων Μεγάλης Κλίμακας

ΔΠΜΣ Επιστήμη Δεδομένων και Μηχανική Μάθηση  
2ο Εξάμηνο

Σταύρος Κοροβέσης  
Αριθμός Μητρώου: 03400297  
Username εργαστηρίου: dsml00297

Ιούνιος 2026

# Περιεχόμενα

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Εισαγωγή</b>                                                  | <b>1</b> |
| 1.1      | Περιγραφή Εργασίας                                               | 1        |
| 1.2      | Σύνολα Δεδομένων                                                 | 1        |
| 1.3      | Περιβάλλον Εκτέλεσης                                             | 1        |
| <b>2</b> | <b>Μορφές Αρχείων (Ζητούμενο 1)</b>                              | <b>1</b> |
| 2.1      | CSV και Εναλλακτικές Μορφές                                      | 1        |
| 2.2      | Επιλεγμένη Μορφή και Μετατροπή Δεδομένων                         | 1        |
| 2.3      | Σύγκριση Επίδοσης                                                | 2        |
| <b>3</b> | <b>Query 1 — Εγκλήματα ανά Τμήμα Ημέρας (Ζητούμενο 2)</b>        | <b>2</b> |
| 3.1      | Μεθοδολογία Υλοποίησης                                           | 2        |
| 3.2      | Αποτελέσματα                                                     | 2        |
| 3.3      | Σύγκριση Επίδοσης                                                | 2        |
| <b>4</b> | <b>Query 2 — Κορυφαίοι Μήνες ανά Έτος (Ζητούμενο 3)</b>          | <b>3</b> |
| 4.1      | Μεθοδολογία Υλοποίησης                                           | 3        |
| 4.2      | Σύγκριση Επίδοσης                                                | 3        |
| <b>5</b> | <b>Query 3 — Κατακεφαλήν Εισόδημα ανά Zip Code (Ζητούμενο 4)</b> | <b>3</b> |
| 5.1      | Μεθοδολογία Υλοποίησης                                           | 3        |
| 5.2      | Σύγκριση Επίδοσης                                                | 3        |
| <b>6</b> | <b>Query 4 — Εγκλήματα ανά Αστυνομικό Τμήμα (Ζητούμενο 5)</b>    | <b>4</b> |
| 6.1      | Υλοποίηση                                                        | 4        |
| 6.2      | Κατακόρυφη Κλιμακωσιμότητα (Vertical Scalability)                | 4        |
| 6.3      | Οριζόντια Κλιμακωσιμότητα (Horizontal Scalability)               | 4        |
| 6.4      | Στρατηγικές Join του Catalyst Optimizer                          | 4        |
| <b>7</b> | <b>Στρατηγικές Join — hint &amp;explain (Ζητούμενο 6)</b>        | <b>5</b> |
| 7.1      | Ανάλυση Query 3 (DataFrame API)                                  | 5        |
| 7.2      | Ανάλυση Query 4                                                  | 5        |
| 7.3      | Σύγκριση Στρατηγικών και Συμπεράσματα                            | 5        |
| <b>8</b> | <b>Συμπεράσματα</b>                                              | <b>5</b> |

## Πίνακες

|     |                                                                        |   |
|-----|------------------------------------------------------------------------|---|
| 2.1 | Χρόνος εκτέλεσης Q2 (DataFrame) — CSV έναντι Parquet . . . . .         | 2 |
| 3.1 | Q1: Ποσοστό εγκλημάτων σε δρόμο ανά περίοδο . . . . .                  | 2 |
| 3.2 | Q1: Χρόνοι εκτέλεσης . . . . .                                         | 2 |
| 4.1 | Q2: DataFrame έναντι Spark SQL . . . . .                               | 3 |
| 5.1 | Q3: DataFrame έναντι RDD . . . . .                                     | 3 |
| 6.1 | Q4: Αύξηση cores/memory ανά executor . . . . .                         | 4 |
| 6.2 | Q4: Σταθερός συνολικός πόρος, διαφορετικός αριθμός executors . . . . . | 4 |
| 7.1 | Q3: Επίδραση join hints . . . . .                                      | 5 |
| 7.2 | Q4: Join hints σε cross join . . . . .                                 | 5 |

# 1 Εισαγωγή

## 1.1 Περιγραφή Εργασίας

Η παρούσα εργασία υλοποιεί τέσσερα analytical queries πάνω σε δημόσια δεδομένα εγκληματικότητας και απογραφής του Los Angeles, με εκτέλεση στο Kubernetes cluster του εργαστηρίου (cslab) μέσω `spark-submit`. Τα datasets βρίσκονται στο HDFS path `/data/`. Ο κώδικας είναι διαθέσιμος στο αποθετήριο: <https://github.com/skorovesis/dsml2026-bigdata>.

**Μέθοδος μέτρησης χρόνου:** όπως προτείνει ο πρακτικός οδηγός του μαθήματος, ο χρόνος μετράται εντός της Python με `perf_counter()` γύρω από το τελικό Spark action (`write`). Η τιμή εξάγεται από τα driver logs ως `QUERY_ELAPSED_SECONDS`. Για τις υλοποιήσεις RDD όπου το `collect()` εκτελείται πριν τη μέτρηση, οι χρόνοι υποεκτιμούν το συνολικό κόστος· σημειώνεται ρητά στην ενότητα 3.

## 1.2 Σύνολα Δεδομένων

Τα βασικά σύνολα δεδομένων είναι τα δύο αρχεία εγκληματικότητας για τις περιόδους 2010–2019 και 2020–2025. Τα αντίστοιχα ονόματα αρχείων είναι `LA_Crime_Data_2010_2019.csv` και `LA_Crime_Data_2020_2025.csv`. Επιπλέον, χρησιμοποιούνται συνοδευτικά δεδομένα απογραφής και εισοδήματος για τα ερωτήματα χωρικής και δημογραφικής ανάλυσης.

## 1.3 Περιβάλλον Εκτέλεσης

Οι μετρήσεις έγιναν στο εργαστηριακό περιβάλλον με Apache Spark, αποθήκευση σε HDFS και εκτέλεση μέσω Kubernetes. Οι ρυθμίσεις πόρων αναφέρονται ξεχωριστά σε κάθε ζητούμενο.

# 2 Μορφές Αρχείων (Ζητούμενο 1)

## 2.1 CSV και Εναλλακτικές Μορφές

Το CSV είναι αναγνώσιμο και συμβατό, αλλά απαιτεί parsing, δεν ενσωματώνει schema και είναι αναποτελεσματικό σε συμπίεση. Το Parquet είναι columnar format με ενσωματωμένο schema, συμπίεση και predicate pushdown, κατάλληλο για analytics στο Spark. Εναλλακτικά μπορούν να χρησιμοποιηθούν JSON, ORC και Avro.

## 2.2 Επιλεγμένη Μορφή και Μετατροπή Δεδομένων

Τα αρχεία `LA_Crime_Data_2010_2019.csv` και `LA_Crime_Data_2020_2025.csv` μετατράπηκαν σε Parquet με το script `convert_parquet.py`. Χρόνος μετατροπής: **295.0** s.

## 2.3 Σύγκριση Επίδοσης

**Πίνακας 2.1:** Χρόνος εκτέλεσης Q2 (DataFrame) — CSV έναντι Parquet

| Format  | Χρόνος (s) | Επιτάχυνση |
|---------|------------|------------|
| CSV     | 32.750     | 1.00×      |
| Parquet | 4.247      | 7.72×      |

Το Parquet είναι  $\sim 8\times$  ταχύτερο στο ίδιο query, λόγω μικρότερου I/O, column pruning και αποφυγής CSV parsing. Για μεγάλα batch analytics workloads, η μετάβαση σε Parquet ή ORC είναι συνήθως η βέλτιστη επιλογή.

## 3 Query 1 — Εγκλήματα ανά Τμήμα Ημέρας (Ζητούμενο 2)

### 3.1 Μεθοδολογία Υλοποίησης

Ταξινομούνται τα τμήματα ημέρας (Πρωί, Απόγευμα, Βράδυ, Νύχτα) κατά φθίνουσο ποσοστό εγκλημάτων με `Premis Desc = "STREET"`. Η εκτέλεση έγινε με 2 executors, 1 core ανά executor και 2GB μνήμης.

### 3.2 Αποτελέσματα

**Πίνακας 3.1:** Q1: Ποσοστό εγκλημάτων σε δρόμο ανά περίοδο

| Περίοδος | street_pct (%) | Σύνολο εγκλημάτων | Σε δρόμο |
|----------|----------------|-------------------|----------|
| Νύχτα    | 28.854         | 870235            | 251094   |
| Βράδυ    | 27.552         | 719695            | 198292   |
| Πρωί     | 18.881         | 693099            | 130866   |
| Απόγευμα | 18.294         | 855099            | 156432   |

Η Νύχτα (21:00–04:59) και το Βράδυ (17:00–20:59) εμφανίζουν το υψηλότερο ποσοστό εγκλημάτων σε δρόμο.

### 3.3 Σύγκριση Επίδοσης

**Πίνακας 3.2:** Q1: Χρόνοι εκτέλεσης

| API             | Script       | Χρόνος (s) |
|-----------------|--------------|------------|
| RDD             | q1_rdd.py    | 1.92       |
| DataFrame       | q1_df.py     | 21.62      |
| DataFrame + UDF | q1_df_udf.py | 224.56     |

Το DataFrame API είναι ταχύτερο από το UDF, καθώς το UDF εμποδίζει το vectorized execution. Ο χρόνος RDD φαίνεται χαμηλός επειδή η μέτρηση ξεκινά μετά το `collect()`. ο Catalyst optimizer στο DataFrame εκτελεί τα aggregations πιο αποδοτικά στο cluster.

## 4 Query 2 — Κορυφαίοι Μήνες ανά Έτος (Ζητούμενο 3)

### 4.1 Μεθοδολογία Υλοποίησης

Το ερώτημα υπολογίζει τους τρεις μήνες με τα περισσότερα εγκλήματα ανά έτος. Οι υλοποιήσεις DataFrame API και Spark SQL εκφράζουν το ίδιο λογικό σχέδιο: ομαδοποίηση ανά έτος και μήνα, `RANK()` ανά έτος και φίλτρο για τις τρεις πρώτες θέσεις. Η εκτέλεση έγινε με 4 executors, 1 core ανά executor και 2GB μνήμης.

### 4.2 Σύγκριση Επίδοσης

Πίνακας 4.1: Q2: DataFrame έναντι Spark SQL

| Υλοποίηση     | Script    | Χρόνος (s) |
|---------------|-----------|------------|
| DataFrame API | q2_df.py  | 327.66     |
| Spark SQL     | q2_sql.py | 101.97     |

Το Spark SQL είναι  $\sim 3.2\times$  ταχύτερο, πιθανότατα λόγω βελτιστοποίησης του Catalyst σε window aggregations και λιγότερου Python overhead στον driver.

## 5 Query 3 — Κατακεφαλήν Εισόδημα ανά Zip Code (Ζητούμενο 4)

### 5.1 Μεθοδολογία Υλοποίησης

Ο δείκτης ορίζεται ως:

$$\text{per\_capita} = \frac{\text{median\_household\_income}_{2021}}{\sum \text{POP}_{20\text{ανά ZCTA}20}}$$

Η εκτέλεση έγινε με 3 executors, 1 core ανά executor και 2GB μνήμης.

### 5.2 Σύγκριση Επίδοσης

Πίνακας 5.1: Q3: DataFrame έναντι RDD

| Υλοποίηση | Script    | Χρόνος (s) |
|-----------|-----------|------------|
| DataFrame | q3_df.py  | 285.30     |
| RDD       | q3_rdd.py | 3.39       |

Το DataFrame περιλαμβάνει ανάγνωση GeoJSON, explode features και join με income. Το RDD script μετρά μόνο το write μετά από in-memory collect(). ο πραγματικός χρόνος RDD είναι σημαντικά μεγαλύτερος. Το DataFrame API είναι προτιμητέο για joins και aggregations σε structured data.

## 6 Query 4 — Εγκλήματα ανά Αστυνομικό Τμήμα (Ζητούμενο 5)

### 6.1 Υλοποίηση

Χρησιμοποιείται cross join εγκλημάτων (LAT/LON) με 21 αστυνομικά τμήματα, με broadcast του μικρού πίνακα τμημάτων. Στη συνέχεια υπολογίζεται η απόσταση Haversine, επιλέγεται το ελάχιστο ανά έγκλημα και γίνεται aggregation ανά DIVISION.

### 6.2 Κατακόρυφη Κλιμακωσιμότητα (Vertical Scalability)

Πίνακας 6.1: Q4: Αύξηση cores/memory ανά executor

| Cores/exec | Μνήμη | Χρόνος (s) |
|------------|-------|------------|
| 1          | 2GB   | 295.14     |
| 2          | 4GB   | 356.36     |
| 4          | 8GB   | 89.38      |

Με 4 cores και 8GB ανά executor παρατηρείται σημαντική βελτίωση ( $\sim 3.3\times$  έναντι 1 core/2GB), καθώς το cross join και οι UDF distances ωφελούνται από περισσότερους παράλληλους workers εντός κάθε executor.

### 6.3 Οριζόντια Κλιμακωσιμότητα (Horizontal Scalability)

Πίνακας 6.2: Q4: Σταθερός συνολικός πόρος, διαφορετικός αριθμός executors

| Ρύθμιση                   | Χρόνος (s)    | Σχόλιο                       |
|---------------------------|---------------|------------------------------|
| 2 exec $\times$ (4c, 8GB) | 218.82        | λιγότερο scheduling overhead |
| 4 exec $\times$ (2c, 4GB) | <b>176.56</b> | <b>καλύτερο</b>              |
| 8 exec $\times$ (1c, 2GB) | 230.81        | περισσότερο network shuffle  |

Η διάταξη  $4 \times (2 \text{ cores}, 4\text{GB})$  δίνει το καλύτερο αποτέλεσμα: ισορροπία μεταξύ παραλληλισμού και μεγέθους JVM heap ανά executor.

### 6.4 Στρατηγικές Join του Catalyst Optimizer

Στο cross join με 21 stations, ο optimizer επιλέγει BroadcastNestedLoopJoin, καθώς ο πίνακας των stations είναι μικρός και μπορεί να γίνει broadcast.

## 7 Στρατηγικές Join — hint & explain (Ζητούμενο 6)

### 7.1 Ανάλυση Query 3 (DataFrame API)

Πίνακας 7.1: Q3: Επίδραση join hints

| Hint                 | Φυσικό plan             | Χρόνος (s)   |
|----------------------|-------------------------|--------------|
| broadcast            | BroadcastHashJoin       | 41.61        |
| merge                | SortMergeJoin           | <b>18.72</b> |
| shuffle_hash         | ShuffledHashJoin        | 47.86        |
| shuffle_replicate_nl | BroadcastNestedLoopJoin | 81.77        |

Ο πίνακας income είναι μικρός (~300 zip codes). Το merge (SortMergeJoin) αποδίδει καλύτερα όταν και οι δύο πλευρές είναι ήδη μεγάλες μετά το aggregation των census blocks.

### 7.2 Ανάλυση Query 4

Πίνακας 7.2: Q4: Join hints σε cross join

| Hint                 | Φυσικό plan             | Χρόνος (s)    |
|----------------------|-------------------------|---------------|
| broadcast            | BroadcastNestedLoopJoin | 294.64        |
| merge                | BroadcastNestedLoopJoin | 294.91        |
| shuffle_hash         | BroadcastNestedLoopJoin | 282.84        |
| shuffle_replicate_nl | BroadcastNestedLoopJoin | <b>271.35</b> |

Στο cross join με 21 stations, η διαφορά μεταξύ hints είναι μικρή (~8%). Το bottleneck είναι ο υπολογισμός αποστάσεων σε ~3 εκατομμύρια εγκλήματα.

### 7.3 Σύγκριση Στρατηγικών και Συμπεράσματα

Τα join hints έχουν ουσιαστικό αντίκτυπο στο Q3, όπου υπάρχουν διαφορετικές εφαρμόσιμες στρατηγικές join. Αντίθετα, στο Q4 ο μικρός πίνακας των αστυνομικών τμημάτων οδηγεί σχεδόν πάντα σε BroadcastNestedLoopJoin.

## 8 Συμπεράσματα

- Το Parquet υπερέρχει σαφώς του CSV για analytics, με επιτάχυνση 7.7× στο Q2.
- Οι υλοποιήσεις DataFrame και SQL προτιμώνται έναντι RDD και UDF για σύνθετα ερωτήματα.
- Στο Q4, η οριζόντια κλιμάκωση με 4 executors × 2 cores δίνει το καλύτερο αποτέλεσμα.
- Τα join hints είναι χρήσιμα στο Q3, ενώ έχουν μικρότερη επίδραση στο cross join του Q4.